

LAB REPORT

Title:-

CS3101: Operating System Lab 4 – Process Handling

Name: Parth Singla

Roll Number: 2401CS18

Objective

The primary objectives of this lab are to understand and implement the following operating system concepts:

- **Process Creation and Hierarchy:** To dynamically construct a specific multi-level process hierarchy using `fork( )` and synchronize parent-child execution using `waitpid( )`.
- **Program Replacement:** To demonstrate process image replacement by combining `fork( )` with the `exec( )` family of functions, and to correctly capture the child's exit status.
- **Process Lifecycle States:** To artificially induce and observe special process states, specifically **Orphan processes** (where the parent terminates before the child) and **Zombie processes** (where the child terminates before the parent collects its status).
- **Concurrent Execution and Monitoring:** To build a process supervisor that creates multiple independent worker processes, allows them to run concurrently, and dynamically monitors their completion order using non-blocking synchronization (`WNOHANG`).

Task 1:

Analysis Requirements:-

1. The total number of processes that should be created. A total of **6** logical processes will be running. This includes the original process (P0) and the 5 created child processes (P1, P2, P3, P4, P5).

2. The parent of each process in the specified hierarchy. Based on the process tree:

- **P0:** The terminal/shell (the original process).
- **P1:** P0
- **P2:** P0
- **P3:** P1
- **P4:** P1
- **P5:** P2

3. The number of fork() calls executed by each logical process.

- **P0** executes **2** fork() calls (to create P1 and P2).
- **P1** executes **2** fork() calls (to create P3 and P4).
- **P2** executes **1** fork() call (to create P5).
- **P3, P4, and P5** execute **0** fork() calls (they are leaf nodes in the hierarchy).

4. Which processes can execute concurrently. All six processes (**P0, P1, P2, P3, P4, and P5**) can execute their computational workload (the simulated sleep() phase) concurrently. Once a child process is created via fork(), it runs in parallel with its parent and siblings.

5. Which processes must terminate before their parent can print its completion message. Because parents must use waitpid() to wait for all their *direct* children:

- **P3** and **P4** must terminate before **P1** can print its completion message.
- **P5** must terminate before **P2** can print its completion message.
- **P1** and **P2** must terminate before **P0** can print its final completion message.

task1.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

void simulate_work_and_exit(const char* name, int sleep_time, int children_count) {
    printf("%s: PID = %d, PPID = %d\n", name, getpid(), getppid());
    sleep(sleep_time); // Simulate work
    printf("%s: Computation completed.\n", name);

    // Wait for all direct children
    for (int i = 0; i < children_count; i++) {
        waitpid(-1, NULL, 0);
    }

    if (children_count > 0) {
        printf("%s: All children have terminated.\n", name);
    }
    exit(0);
}

int main() {
    printf("P0: PID = %d, PPID = %d\n", getpid(), getppid());

    pid_t p1 = fork();
    if (p1 == 0) {
        // Inside P1
        pid_t p3 = fork();
        if (p3 == 0) simulate_work_and_exit("P3", 1, 0);

        pid_t p4 = fork();
        if (p4 == 0) simulate_work_and_exit("P4", 2, 0);

        simulate_work_and_exit("P1", 3, 2);
    } else if (p1 > 0) {
        pid_t p2 = fork();
        if (p2 == 0) {
            // Inside P2
            pid_t p5 = fork();
            if (p5 == 0) simulate_work_and_exit("P5", 1, 0);

            simulate_work_and_exit("P2", 2, 1);
        } else if (p2 > 0) {
            // Inside P0
            sleep(4); // P0 does its own work
            printf("P0: Computation completed.\n");
            waitpid(p1, NULL, 0);
            waitpid(p2, NULL, 0);
            printf("P0: All processes have terminated.\n");
        }
    } else {
        perror("Fork failed");
        exit(1);
    }
    return 0;
}
```

```

● parthsingla@172-9-9-227 2401CS18_05lab4 % ./a.out
P0: PID = 20700, PPID = 20113
P3: PID = 20705, PPID = 20703
P2: PID = 20704, PPID = 20700
P1: PID = 20703, PPID = 20700
P5: PID = 20706, PPID = 20704
P4: PID = 20707, PPID = 20703
P3: Computation completed.
P5: Computation completed.
P2: Computation completed.
P4: Computation completed.
P2: All children have terminated.
P1: Computation completed.
P1: All children have terminated.
P0: Computation completed.
P0: All processes have terminated.
○ parthsingla@172-9-9-227 2401CS18_05lab4 % █

```

Task 1 output

Task 2:

Execution and Observation :-

1. Record the PID of the parent and child.

- **Parent PID:** 21103
- **Child PID:** 21106

2. Verify that the PID of the child remains the same when the child program is replaced using `exec()`.

- **Verified:** The output shows the child's PID was 21106 before calling `exec ( )`, and the new program (`task2_exec`) also printed PID 21106. The process identity did not change.

3. Record the PID printed by the child immediately before the `exec()` call.

- **PID:** 21106

4. Record the PID printed by `task2_exec` after the `exec()` call.

- **PID:** 21106

5. Verify experimentally that: PID before `exec` = PID after `exec`.

- **Verified:** $21106 = 21106$. This experimentally proves that the `exec ( )` system call replaces the memory space and process image of the current process with a new one, but it **does not** create a new process or change the Process ID.

6. Verify that the parent prints its final completion message only after `task2_exec` has terminated.

- **Verified:** The parent process output (Parent: Child 21106 terminated normally. and Parent: Child execution completed.) appeared *after* the Exec program: PID = 21106 line. This confirms that `waitpid ( )` successfully blocked the parent until `task2_exec` finished its execution.

7. Modify the child so that it attempts to execute a program that does not exist. Observe the behavior when `exec()` fails and report the exit status observed by the parent.

- *(Since you haven't run this specific modification yet, here is what happens based on the C code we wrote):*
- If you change the `execl` call to a non-existent file (e.g., `execl( ". / does_not_exist", ... )`), the `exec( )` call will fail and return to the original child code.
- The `perror( "Exec failed" );` line will execute, printing an error like: **Exec failed: No such file or directory.**
- The child will then hit `exit( 1 );`.
- **Reported Exit Status:** The parent process will catch this and print: **Parent: Child exit status = 1.**

task2_exec.c

```
#include <stdio.h>

#include <unistd.h>

int main() {

    printf("Exec program: PID = %d\n", getpid());

    return 0; // Terminate normally

}
```

task2_parent.c

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>

int main() {

    printf("Parent: PID = %d\n", getpid());

    pid_t pid = fork();
```

```

if (pid < 0) {
    perror("Fork failed");
    exit(1);
} else if (pid == 0) {
    printf("Child: PID = %d\n", getpid());
    printf("Child: PPID = %d\n", getppid());

    execl("./task2_exec", "task2_exec", NULL);

    perror("Exec failed");
    exit(1);
} else {
    int status;

    waitpid(pid, &status, 0);

    if (WIFEXITED(status)) {
        printf("Parent: Child %d terminated normally.\n", pid);
        printf("Parent: Child exit status = %d\n", WEXITSTATUS(status));
    } else {
        printf("Parent: Child %d terminated abnormally.\n", pid);
    }

    printf("Parent: Child execution completed.\n");
}

return 0;
}

```

```

● parthsingla@172-9-9-227 task2 % gcc -Wall -Wextra -std=c11 task2_exec.c -o task2_exec
● parthsingla@172-9-9-227 task2 % gcc -Wall -Wextra -std=c11 task2_parent.c -o task2_parent
● parthsingla@172-9-9-227 task2 % ./task2_parent
Parent: PID = 21103
Child: PID = 21106
Child: PPID = 21103
Exec program: PID = 21106
Parent: Child 21106 terminated normally.
Parent: Child exit status = 0
Parent: Child execution completed.
○ parthsingla@172-9-9-227 task2 % █

```

Task 2 output

Task 3:

Observations:-

Part A: Orphan-Process Experiment

- **Parent PID:** 21395
- **Child PID:** 21396
- **Child PPID before the parent terminates:** 21395
- **Child PPID after the parent terminates:** 1 (*The process was successfully adopted by the init process, which always has PID 1*).
- **The process state of the child while it continues executing:** Since the child is waiting for the parent to die (via `sleep ( )`), it will be in an **S (Interruptible Sleep)** state. Once it wakes up to print its final statement, it temporarily enters an **R (Running)** state.

Part B: Zombie-Process Experiment

- **Parent PID:** 22272
- **Child PID:** 22273
- **Child's termination status:** 42
- **Process state observed before `waitpid ( )`:** **Z+** alongside the **<defunct>** marker, confirming it successfully became a zombie.
- **Whether the child's process entry still exists after `waitpid ( )` returns:** **No.** As shown in "image.png", the parent successfully collects the termination status and removes the zombie from the system process table.

task3_orphan.c

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

int main() {

    int fd[2];

    if (pipe(fd) == -1) { perror("Pipe failed"); exit(1); }
```

```

pid_t pid = fork();

if (pid > 0) {

    printf("Parent: PID = %d\n", getpid());

    printf("Parent: Child PID = %d\n", pid);


    char buf;

    read(fd[0], &buf, 1); // Wait for child notification

    printf("Parent: Terminating now.\n");

    exit(0); // Parent terminates without waiting

} else if (pid == 0) {

    printf("Child: PID = %d, PPID = %d\n", getpid(), getppid());

    write(fd[1], "K", 1); // Notify parent


    sleep(2); // Wait for parent to die

    printf("Child: After parent termination\n");

    printf("Child: PID = %d, PPID = %d\n", getpid(), getppid());

    exit(0);

}

return 0;

}

```

task3_zombie.c

```

#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>

```

```

int main() {

    pid_t pid = fork();


    if (pid > 0) {

        printf("Parent: PID = %d\n", getpid());

```

```

printf("Parent: Child PID = %d\n", pid);

printf("Parent: Waiting before calling waitpid().\n");

sleep(10); // Deliberate delay to create a zombie

int status;

waitpid(pid, &status, 0);

if (WIFEXITED(status)) {

    printf("Parent: Child terminated normally.\n");

    printf("Parent: Child exit status = %d\n", WEXITSTATUS(status));

    printf("Parent: Child termination status collected; zombie removed.\n");

}

} else if (pid == 0) {

    printf("Child: PID = %d, PPID = %d\n", getpid(), getppid());

    printf("Child: Terminating now.\n");

    exit(42); // Explicit termination status

}

return 0;

}

```

```

● parthsingla@172-9-9-227 task3 % gcc -Wall -Wextra -std=c11 task3_orphan.c -o task3_orphan
● parthsingla@172-9-9-227 task3 % ./task3_orphan
Parent: PID = 21395
Parent: Child PID = 21396
Child: PID = 21396, PPID = 21395
Parent: Terminating now.
○ parthsingla@172-9-9-227 task3 % Child: After parent termination
Child: PID = 21396, PPID = 1

```

```

● parthsingla@172-9-9-227 task3 % ./task3_zombie
Parent: PID = 22272
Parent: Child PID = 22273
Parent: Waiting before calling waitpid().
Child: PID = 22273, PPID = 22272
Child: Terminating now.
Parent: Child terminated normally.
Parent: Child exit status = 42
Parent: Child termination status collected; zombie removed.
○ parthsingla@172-9-9-227 task3 %

```

```
[parthsingla@172-9-9-227 task3 % ps -o pid,ppid,stat,command -p 22273
  PID  PPID STAT COMMAND
22273 22272 Z+    <defunct>
parthsingla@172-9-9-227 task3 %
```

Task 3 output

Task 4:

worker1.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void merge(int arr[], int l, int m, int r) {
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;
    int *L = malloc(n1 * sizeof(int));
    int *R = malloc(n2 * sizeof(int));

    for (i = 0; i < n1; i++) L[i] = arr[l + i];
    for (j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    i = 0; j = 0; k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) { arr[k] = L[i]; i++; }
        else { arr[k] = R[j]; j++; }
        k++;
    }
    while (i < n1) { arr[k] = L[i]; i++; k++; }
    while (j < n2) { arr[k] = R[j]; j++; k++; }
    free(L); free(R);
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

int main() {
    printf("Worker 1: PID = %d, PPID = %d\n", getpid(), getppid());
    int size = 1000000;
    int *arr = malloc(size * sizeof(int));

    // Initialize array with random values
    for (int i = 0; i < size; i++) arr[i] = rand() % 1000;

    mergeSort(arr, 0, size - 1);

    // Verify sorting
    int is_sorted = 1;
    for (int i = 0; i < size - 1; i++) {
        if (arr[i] > arr[i+1]) { is_sorted = 0; break; }
    }
}
```

```

}

printf("Worker 1: Sorting completed.\n");
printf("Worker 1: Sorted = %s\n", is_sorted ? "YES" : "NO");
printf("Worker 1: First five values = %d %d %d %d %d\n", arr[0], arr[1], arr[2], arr[3], arr[4]);

free(arr);
return 0;
}

```

worker2.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int binarySearch(int arr[], int l, int r, int x) {
    while (l <= r) {
        int m = l + (r - l) / 2;
        if (arr[m] == x) return m;
        if (arr[m] < x) l = m + 1;
        else r = m - 1;
    }
    return -1;
}

int main() {
    printf("Worker 2: PID = %d, PPID = %d\n", getpid(), getppid());

    int size = 100000;
    int *arr = malloc(size * sizeof(int));

    // Initialize a sorted array
    for (int i = 0; i < size; i++) arr[i] = i * 2;

    int success_count = 0;
    int queries = 10000000;

    // Perform 10,000,000 searches
    for(int i = 0; i < queries; i++) {
        int target = rand() % (size * 2); // Some will exist, some won't
        if (binarySearch(arr, 0, size - 1, target) != -1) {
            success_count++;
        }
    }

    printf("Worker 2: Search completed.\n");
    printf("Worker 2: Successful searches = %d\n", success_count);

    free(arr);
    return 0;
}

```

worker3.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

struct Node {
    int data;
    struct Node* left;
    struct Node* right;
}

```

```

};

struct Node* newNode(int data) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = data;
    node->left = NULL;
    node->right = NULL;
    return node;
}

struct Node* insert(struct Node* node, int data) {
    if (node == NULL) return newNode(data);
    if (data < node->data) node->left = insert(node->left, data);
    else if (data > node->data) node->right = insert(node->right, data);
    return node;
}

void preorder(struct Node* root, int* count) {
    if (root != NULL) {
        (*count)++;
        preorder(root->left, count);
        preorder(root->right, count);
    }
}

void postorder(struct Node* root, int* count) {
    if (root != NULL) {
        postorder(root->left, count);
        postorder(root->right, count);
        (*count)++;
    }
}

void freeTree(struct Node* root) {
    if (root != NULL) {
        freeTree(root->left);
        freeTree(root->right);
        free(root);
    }
}

int main() {
    printf("Worker 3: PID = %d, PPID = %d\n", getpid(), getppid());

    struct Node* root = NULL;
    int num_nodes = 100000;

    // Construct tree
    for (int i = 0; i < num_nodes; i++) {
        root = insert(root, rand());
    }

    int pre_count = 0, post_count = 0;

    preorder(root, &pre_count);
    printf("Worker 3: Preorder traversal completed.\n");

    postorder(root, &post_count);
    printf("Worker 3: Postorder traversal completed.\n");

    printf("Worker 3: Preorder nodes = %d\n", pre_count);
    printf("Worker 3: Postorder nodes = %d\n", post_count);

    freeTree(root);
    return 0;
}

```

task4.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    printf("Parent: PID = %d\n", getpid());
    pid_t children[3];
    char *programs[] = { "./worker1", "./worker2", "./worker3" };

    for (int i = 0; i < 3; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            printf("Child %d before exec: PID = %d, PPID = %d\n", i+1, getpid(), getppid());
            execl(programs[i], programs[i] + 2, NULL);
            perror("Exec failed");
            exit(1);
        } else if (pid > 0) {
            children[i] = pid;
        }
    }

    int completed = 0;
    pid_t completion_order[3];
    int exit_statuses[3];
    int worker_ids[3];

    while (completed < 3) {
        int status;
        pid_t p = waitpid(-1, &status, WNOHANG); // Non-blocking monitoring[cite: 1]

        if (p > 0) {
            int wid = -1;
            for(int i=0; i<3; i++) if (children[i] == p) wid = i + 1;

            completion_order[completed] = p;
            worker_ids[completed] = wid;

            if (WIFEXITED(status)) {
                exit_statuses[completed] = WEXITSTATUS(status);
                printf("Parent: worker%d terminated.\n", wid);
                printf("Parent: PID = %d\nParent: Normal termination.\nParent: Exit status = %d\n", p, exit_statuses[completed]);
            }
            completed++;
        } else {
            sleep(1); // Avoid busy waiting
        }
    }

    printf("\n===== \n");
    printf("Parent: Final Completion Order\n");
    printf("===== \n");
    for (int i = 0; i < 3; i++) {
        printf("%d. worker%d (PID = %d), exit status = %d\n", i+1, worker_ids[i], completion_order[i], exit_statuses[i]);
    }
    printf("\nParent: All children have terminated.\n");
    return 0;
}
```

```

● parthsingla@172-9-9-227 task4 % gcc -Wall -Wextra -std=c11 worker1.c -o worker1
gcc -Wall -Wextra -std=c11 worker2.c -o worker2
gcc -Wall -Wextra -std=c11 worker3.c -o worker3
● parthsingla@172-9-9-227 task4 % gcc -Wall -Wextra -std=c11 task4.c -o task4
● parthsingla@172-9-9-227 task4 % ./task4
Parent: PID = 23465
Child 1 before exec: PID = 23468, PPID = 23465
Child 2 before exec: PID = 23469, PPID = 23465
Child 3 before exec: PID = 23470, PPID = 23465
Worker 1: PID = 23468, PPID = 23465
Worker 1: Sorting completed.
Worker 1: Sorted = YES
Worker 1: First five values = 0 0 0 0 0
Worker 2: PID = 23469, PPID = 23465
Parent: worker1 terminated.
Parent: PID = 23468
Parent: Normal termination.
Parent: Exit status = 0
Worker 3: PID = 23470, PPID = 23465
Worker 3: Preorder traversal completed.
Worker 3: Postorder traversal completed.
Worker 3: Preorder nodes = 100000
Worker 3: Postorder nodes = 100000
Parent: worker3 terminated.
Parent: PID = 23470
Parent: Normal termination.
Parent: Exit status = 0
Worker 2: Search completed.
Worker 2: Successful searches = 4998503
Parent: worker2 terminated.
Parent: PID = 23469
Parent: Normal termination.
Parent: Exit status = 0

=====
Parent: Final Completion Order
=====
1. worker1 (PID = 23468), exit status = 0
2. worker3 (PID = 23470), exit status = 0
3. worker2 (PID = 23469), exit status = 0

Parent: All children have terminated.
○ parthsingla@172-9-9-227 task4 % █

```

Task 4 output

Results and Discussion

The assignment was successfully executed across three progressively complex tasks, with all edge cases and synchronization requirements met.

- **Task 1:** A dynamically allocated, multi-level process hierarchy was successfully generated. The output confirmed that parent processes effectively blocked their execution using `waitpid()`, strictly ensuring that all child processes terminated before their respective parents.
- **Task 2:** Process image replacement was executed successfully. The observed PID of the child process remained perfectly identical before and after invoking the `exec()` system call, and the parent correctly captured the normal exit status of the replaced program.
- **Task 3:** The lifecycle of edge-case processes was verified. The orphan process's PPID changed to 1 after its parent terminated, proving adoption by the system's `init` process.

The zombie process temporarily entered a `<defunct>` (Z+) state in the system table until the delayed `waitpid()` call successfully reaped it.

- **Task 4:** Three computationally intensive worker programs (Merge Sort, Binary Search, and Tree Traversal) executed concurrently. The supervisor parent successfully tracked their asynchronous termination order using a non-blocking wait mechanism

Conclusion

This laboratory assignment successfully demonstrated the fundamental principles of process management in a Linux operating system. By utilizing core POSIX system calls such as `fork()`, `exec()`, and `waitpid()`, the practical mechanics of process creation, hierarchical structuring, and synchronization were thoroughly explored.

The experiments highlighted the critical importance of parent-child synchronization to prevent resource leaks, such as zombie processes, and illustrated how the operating system safely handles orphan processes by reassigning them. Furthermore, the successful implementation of a multi-process supervisor executing distinct algorithmic workloads emphasized the power of concurrent execution and non-blocking state monitoring.

Ultimately, this lab provided essential hands-on experience in system-level programming, reinforcing the theoretical concepts of process lifecycles, memory isolation, and asynchronous task management necessary for building robust operating system applications.